

This JavaScript Code will create a diagonal line. Let us see how this code works:

- ▶ First of all you gain access to the canvas object by referencing a particular canvas's ID value. In this example, the ID is diagonal.
- ▶ Next, you create a context variable and you call the canvas object's `getContext` method, passing in the type of canvas you are looking for. You pass in the string "2d" to get a two-dimensional context.
- ▶ You then use the context to perform drawing operations. In this case, you can create the diagonal line by calling three methods—`beginPath`, `moveTo`, and `lineTo`—passing in the start and end coordinates of the diagonal.
- ▶ The `moveTo()` method creates a new subpath for the given point which becomes the new context point. You can think of the `moveTo()` method as a way to position your drawing cursor.
- ▶ The `lineTo()` method draws a line from the context point to the given point.
- ▶ The drawing methods `moveTo` and `lineTo` do not actually create the line; you finalize a canvas operation and draw the line by calling the `context.stroke()`; method
- ▶ The `stroke()` method assigns a color to the line and makes it visible. Unless otherwise specified, the default stroke color is black.

Almost all the important functions with visual output are accessible only from the context. This flexibility enables the canvas to support different types of drawing models in the future, based on the type of context that is retrieved from the canvas.

As we saw in the previous example, many operations on the context do not immediately update the drawing surface. Functions such as `beginPath`, `moveTo`, and `lineTo` do not modify the canvas appearance immediately. The same is true of many functions that set the styling and preferences of the canvas. Only when a path is stroked or filled does it appear on the display. Otherwise, the canvas will only be immediately updated when images are displayed or rectangles are drawn, filled, or cleared.

Applying Transformations

Now let's look at another way to draw on the canvas using transformation. In the following example, the result is identical to the previous example, but the code used to draw the diagonal line is different. Transformation can be thought of as a best practice for more complex canvas operations as it is critical to understanding the HTML5 Canvas API's complex capabilities.